

Overview

COTA Project

ISE 3230

Group 11

Tony Tran, Wren Lanzendorfer, Kathryn Coblentz

Introduction/Motivation:

The Central Ohio Transit Authority (COTA) operates the public bus system in Columbus, Ohio. While the system covers most of the city, some residential and employment-dense areas remain underserved, with limited walking access to existing bus stops. Our project's aim was to determine optimal locations for new COTA bus stops to improve service accessibility for underserved regions while considering factors such as walking distance, population density, and existing route connectivity.

This problem has real-world implications for urban planning and sustainability, with a scope including all of COTA's regular bus routes excluding special express and commuter lines.

Model Description:

Summary: Our objective is to place a limited number of bus stops that will cover the greatest population possible within Central Ohio where there is not a bus stop in range (0.5 miles or ~805 meters). We define the number of bus stops considered as p , where p is 5 (in other words, we will look into where the top 5 potential bus stops could be made that will cover the most population). We define the coverage as within 0.5 miles of a given stop. Our model works by looking at which candidate locations to build stops at as well as demand for those given stops.

```
# ----- MILP Variables -----  
MAX_NEW_STOPS = 5      # p (baseline)  
D_MAX = 805           # meters (baseline)
```

In our model, we first need to determine where stops can be placed. Using data from GTFS (data of all existing COTA stops), we create a set of possible locations, which we will call the candidate location (denoted as $\mathbf{C}$). The candidate location is defined as a binary variable $\mathbf{x}_j$, where $\mathbf{x}_j = 1$ if a stop is built at candidate j ; 0 otherwise. $j = 1, \dots, m$

Decision variables

```
x = model.addMVar(m, vtype=GRB.BINARY, name="x")
```

```
candidates = underserved.reset_index(drop=True).copy()
candidates = gpd.GeoDataFrame(candidates, geometry="geometry", crs="EPSG:4326")
m_cand = len(candidates)
print(f"    Candidate stop locations: {m_cand}")
if m_cand == 0:
    print("No candidate locations (no underserved tracts). Exiting.")
    return

candidates_m = candidates.to_crs(CRS_METRIC)
demand_m = demand.to_crs(CRS_METRIC)
```

Next, we need to determine whether each demand point is covered. We will create a set of demand points (denoted as **D**) that represent population centers (based off the TIGER Tracts city lines). The demand point is defined as a binary variable y_i , where $y_i = 1$ if demand point i is covered; 0 otherwise. $i = 1, \dots, n$. We consider a demand point as “covered” if there is at least one bus stop that exists within a $\frac{1}{2}$ mile radius.

```
y = model.addMVar(n, vtype=GRB.BINARY, name="y")
```

```
demand = tracts.copy()
demand["geometry"] = centroids_wgs84.values
demand["lat"] = demand.geometry.y
demand["lon"] = demand.geometry.x
demand = gpd.GeoDataFrame(demand, geometry="geometry", crs="EPSG:4326")
n_dem = len(demand)
print(f"    Demand points (tract centroids): {n_dem}")

stops_m = stops.to_crs(CRS_METRIC)
demand_m = demand.to_crs(CRS_METRIC)
```

Now that we have our two components, we need to determine if candidate stop j is close enough to demand point i . We denote this as a binary variable a_{ij} , where $a_{ij} = 1$ if candidate j covers demand i ; 0 otherwise.

```

n = len(demand_m)
m = len(candidates_m)
A = np.zeros((n, m), dtype=int)
for i, neigh in enumerate(neighbors):
    if len(neigh) > 0:
        A[i, neigh] = 1
return A

```

```

n, m = A.shape

```

```

print("Building baseline coverage matrix (KDTree)...")
A_baseline = build_coverage_matrix_from_metric(demand_m, candidates_m, D_MAX)

```

Using our newly created relationship between $\mathbf{a}_{ij}$, $\mathbf{y}_i$, and $\mathbf{x}_j$, we will need to ensure that coverage only occurs if a stop is selected. For all demand point i within the set of demand locations $\mathbf{D}$, enforce that the demand point $\mathbf{y}_i \leq \sum_{j \in C} a_{ij} x_j$. This constraint relies on i to ensure we have coverage.

Note: If no stops can cover the demand point i , the RHS is 0, meaning that demand point i isn't covered ($\mathbf{y}_i = 0$). However, if at least one stop can cover demand point i , we allow $\mathbf{y}_i = 1$.

```

# Coverage constraints: y_i <= \sum_j A[i,j] x_j
model.addConstrs((y[i] <= A[i, :] @ x for i in range(n)), name="coverage")

```

Now that we have a reliable model to determine coverage as well as candidate and demand locations, we will limit the amount of new stops that could be built. This is denoted as $\sum_{j \in C} x_j = \mathbf{p}$, where $\mathbf{p}$ represents the number of stops to build. In our case, $\mathbf{p} = 5$, as we want to build 5 new stops.

Finally, we now need to measure the “need” for a new bus stop in a given population. We will use the variable $\mathbf{w}_i$ to denote the “weight” of demand point i . This allows us to

build our objective function, where we aim to maximize the population's need for a new bus stop (within the demand location set **D**) as well as the requirements necessary to

build one. This is written as: $\text{Max } z = \sum_{i \in D} w_i y_i$.

```
# Objective: maximize w*y
model.setObjective(w @ y, GRB.MAXIMIZE)

# Solve
model.optimize()
```

```
for d in D_MAX_LIST:
    for p in MAX_NEW_LIST:
        for e in EQUITY_MULTIPLIERS:
            weights = demand["population"].values * (1.0 + e * density_norm)
```

In short, this is our full model:

$$\begin{aligned}
 &\text{Maximize } z = \sum_{i \in D} w_i y_i \\
 &\text{s.t.} \\
 &y_i \leq \sum_{j \in C} a_{ij} x_j \quad \forall i \in D \\
 &\sum_{j \in C} x_j = p \\
 &x_j \in \{0, 1\} \quad \forall j \in C \\
 &y_i \in \{0, 1\} \quad \forall i \in D
 \end{aligned}$$

where

w_i represents the weight at demand point i

y_i is a binary variable that is 1 if demand point i is covered; 0 otherwise

U represents the set of demand points

a_{ij} is a binary variable that is 1 if candidate j covers demand i ; 0 otherwise

x_j is a binary variable that is 1 if a stop is built at candidate j ; 0 otherwise

$p = 5$

i represents the index of demand points, where $i = 1, \dots, n$

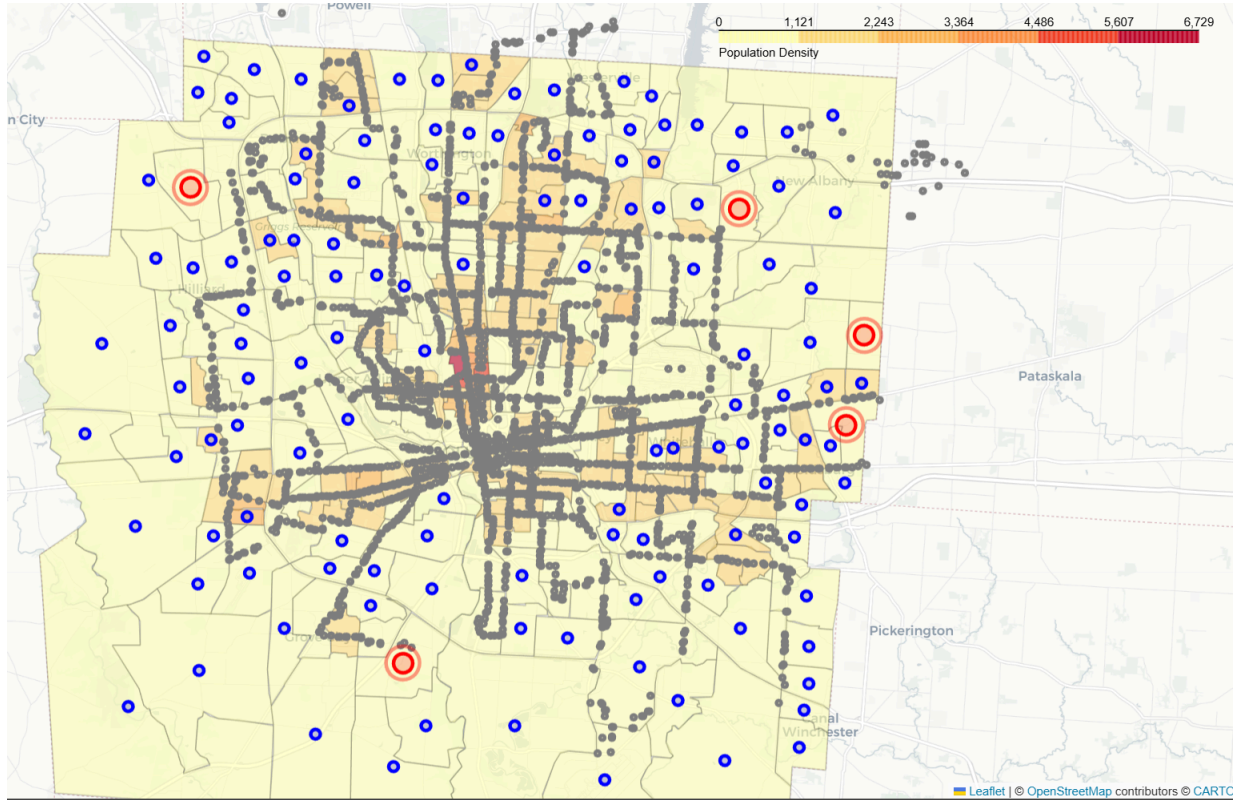
j represents the index of candidates, where $j = 1, \dots, m$

C = Set of Candidate locations

D = Set of Demand locations

Results:

Using our optimization model, we identified five new bus stop locations that maximize population coverage within a one half mile walking distance threshold. The resulting map displays all existing COTA stops as gray points and the selected new stops as red points. We also display other potential bus stops as blue; however, these bus stops aren't considered 'optimal' enough to include in our final interpretations. Around each new stop, a red circular buffer illustrates the half mile coverage radius. The background shading reflects population density across Columbus, which makes it clear how the newly selected stops align with areas that have high residential concentrations but limited access to existing transit service.



The table below summarizes each of the five chosen stops, including GEOID, geographic coordinates, and the population covered by each location. Across all five stops, the model adds 42,986 additional residents to the covered population. This represents a substantial increase relative to the baseline network and demonstrates that a small number of well-placed stops can greatly expand accessibility.

Selected Stop	GEOID	Latitude	Longitude	Population
16	39049009390	39.974217	-82.790753	7768
40	39049006241	40.077974	-83.165966	8591
41	39049007398	40.013312	-82.780411	9389
82	39049009755	39.869836	-83.044463	7639
130	39049007209	40.068750	-82.852063	9599

Overall these results show that our optimization model fulfills its intended purpose and provides an efficient and data driven approach to expanding the bus network.

Post-optimality analysis:

To understand how sensitive our results are to modeling assumptions, we conducted a series of post-optimality experiments that varied three key parameters: the allowed walking distance, the number of new stops to construct, and the degree to which equity considerations influence the objective.

```
print("\n\n=== POST-OPTIMALITY ANALYSIS: exploring parameters ===")
```

```
D_MAX_LIST = [600, 805, 1000]
```

```
MAX_NEW_LIST = [3, 5, 7]
```

```
EQUITY_MULTIPLIERS = [0.0, 1.0, 2.0]
```

```
for d in D_MAX_LIST:
    for p in MAX_NEW_LIST:
        for e in EQUITY_MULTIPLIERS:
            weights = demand["population"].values * (1.0 + e * density_norm)
```

Changing the number of allowed new stops had the strongest impact on overall coverage. Increasing the stop budget from 3 to 5 to 7 led to substantial increases in total population served, and the optimal stop locations shifted meaningfully across scenarios. This suggests that the stop budget is the most influential decision variable; even a small increase in the number of permitted stops results in significant improvements in accessibility.

To incorporate equity, the model adjusted the weight of each census tract based on its population density. Density serves as a practical proxy for need, since denser neighborhoods tend to have higher concentrations of residents who rely on public transit. Each tract's population density was normalized between 0 and 1, and this value was used to create an equity-adjusted weight using the formula:

$$\text{weight}_i = \text{population}_i \times (1 + \text{equity multiplier} \times \text{normalized density}_i)$$

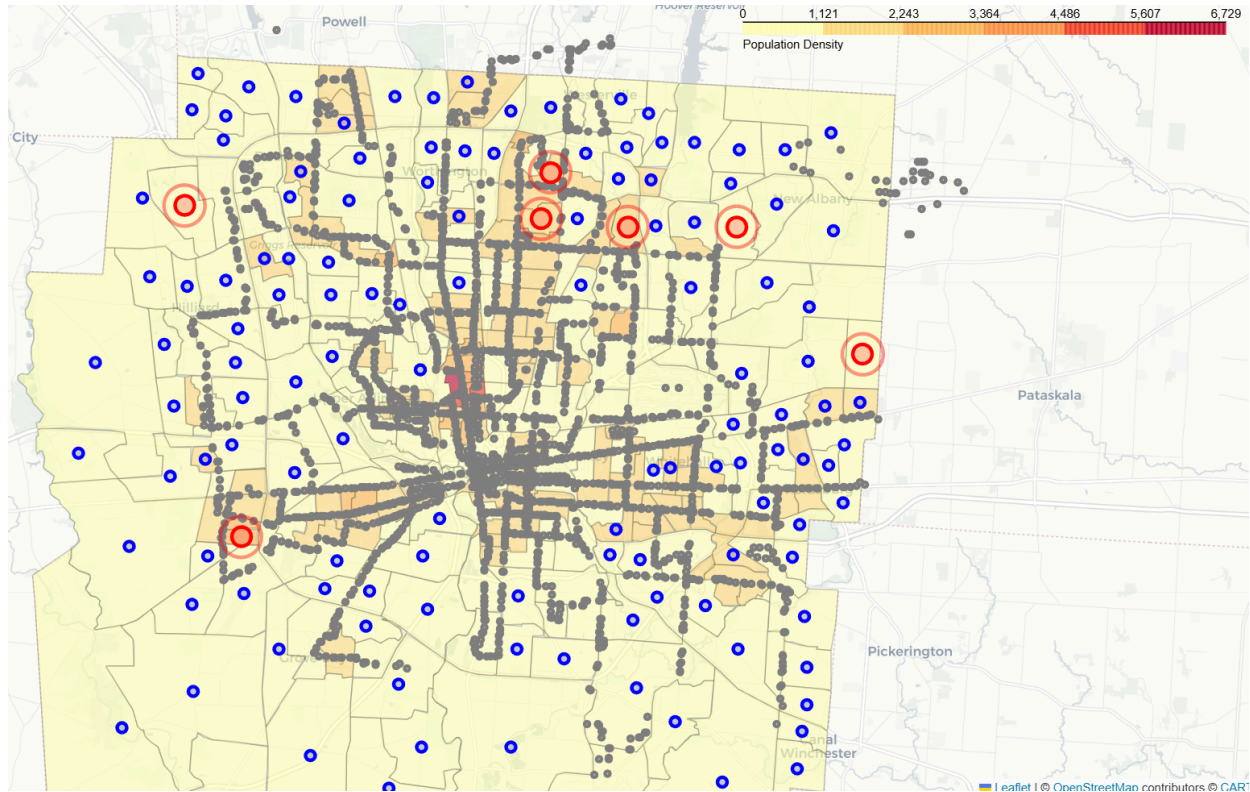
When the equity multiplier increased, the model assigned higher relative importance to dense neighborhoods even if they were not the most populated overall. This ended up shifting the stops more towards the north-east part of the city. Although these adjustments did shift the solution toward higher-need areas, the influence was less

dramatic than the effect of changing the number of stops. Equity weighting refined the prioritization but did not fundamentally alter the structure of the solution.

Lastly, we analyzed the solution when walking distance was changed. Our original model had a radius of 805 meters (or half a mile), assuming that people are willing to walk half a mile to a bus stop. We tested the model by decreasing the radius to 600 meters and by increasing the radius to 1000 meters. These changes did not impact the solution meaningfully. Most of the chosen stops remained the same, even as the radius varied.

After testing all these factors against each other, we found the combination which performed the best was:

Selected Stop	GEOID	Latitude	Longitude	Population
21	39049009390	40.072197	-82.963182	7048
40	39049006241	40.077974	-83.165966	8591
41	39049007398	40.013312	-82.780411	9389
96	39049006945	40.092437	-82.958051	6399
127	39049008164	39.933867	-83.133878	5231
129	39049007199	40.068679	-82.914028	7347
130	39049007209	40.068750	-82.852063	9599



Final Interpretations:

Overall, the combined results show that the model provides a stable and interpretable solution for bus stop placement. While equity weighting and walking distance influence the optimization to some degree, the dominant driver of accessibility improvement is the number of stops constructed. These insights can help planners make budget-conscious decisions while incorporating community needs and spatial equity considerations into transit expansion.

Appendix A. Video

The project video is available at the following link:

[Video Link](#)

[Video Slides](#)

Appendix B. GitHub

All project code is available in the GitHub repository linked below:

[GitHub Repository](#)

Appendix C. Contributions

Each team member participated in the technical aspects of the project, including model formulation and coding.

Tony Tran

- Converted the handwritten model into LaTeX
- Wrote the model description and created the list of decision variables
- Participated in model formulation and coding

Wren Lanzendorfer

- Gathered the data used in the project
- Researched potential modeling approaches
- Wrote the introduction/motivation section
- Participated in model formulation and coding

Kathryn Coblenz

- Created and edited the project video
- Created and organized the GitHub repository
- Wrote the results section
- Participated in model formulation and coding

References/Sources used:

Maximum Coverage Location Problem (MCLP) -- <https://arxiv.org/html/2509.23334>

Appendix

Appendix A. Video

The project video is available at the following link:

[Video Link](#)

[Video Slides](#)

Appendix B. GitHub

All project code is available in the GitHub repository linked below:

[GitHub Repository](#)

Appendix C. Contributions

Each team member participated in the technical aspects of the project, including model formulation and coding.

Tony Tran

- Converted the handwritten model into LaTeX
- Wrote the model description and created the list of decision variables
- Participated in model formulation and coding

Wren Lanzendorfer

- Gathered the data used in the project
- Researched potential modeling approaches
- Wrote the introduction/motivation section
- Participated in model formulation and coding

Kathryn Coblentz

- Created and edited the project video
- Created and organized the GitHub repository
- Wrote the results section
- Participated in model formulation and coding

References/Sources used:

Maximum Coverage Location Problem (MCLP) -- <https://arxiv.org/html/2509.23334>

Code [Gurobi]

```
"""
```

```
Complete pipeline:
```

- downloads GTFS (COTA) stops
- downloads TIGER tracts (Ohio)
- queries ACS for population
- builds candidate stops
- computes distances (Haversine by default)
- outputs CSVs ready for optimization (Gurobi MILP)
- folium map
- post optimality analysis

```
"""
```

```
# ----- IMPORTS -----
```

```
import os
```

```
import zipfile
```

```
import io
```

```
import tempfile
```

```
import webbrowser
```

```
import requests
```

```
import pandas as pd
```

```
import geopandas as gpd
```

```
import numpy as np
```

```
import gurobipy as gp
```

```
from gurobipy import GRB
```

```
from shapely.geometry import Point
```

```
from scipy.spatial import cKDTree
```

```
from tqdm import tqdm
```

```
# ----- Variables & Sources -----
```

```
GTFS_URL = "https://www.cota.com/data/cota.gtfs.zip"
```

```
TIGER_TRACTS_URL =
```

```
"https://www2.census.gov/geo/tiger/TIGER2023/TRACT/tl_2023_39_tract.zip"
```

```
CENSUS_API = "https://api.census.gov/data/2023/acs/acs5"
```

```
ACS_VARIABLE = "B01003_001E" # population
```

```
FRANKLIN_FIPS = ("39", "049")
```

```
# ----- MILP Variables -----
```

```

MAX_NEW_STOPS = 5      # p (baseline)
D_MAX = 805            # meters (baseline)
OUT_DIR = "output"
CRS_METRIC = 3857      # metric CRS for distance queries
os.makedirs(OUT_DIR, exist_ok=True)

# ----- UTILITY FUNCTIONS -----
def download_gtfs_stops(gtfs_url):
    r = requests.get(gtfs_url)
    z = zipfile.ZipFile(io.BytesIO(r.content))
    stops_df = pd.read_csv(z.open("stops.txt"))
    stops_gdf = gpd.GeoDataFrame(
        stops_df,
        geometry=gpd.points_from_xy(stops_df.stop_lon, stops_df.stop_lat),
        crs="EPSG:4326",
    )
    return stops_gdf

def download_tiger_tracts(zip_url, statefp, countyfp):
    r = requests.get(zip_url)
    tmp = tempfile.mkdtemp()
    zip_path = os.path.join(tmp, "tracts.zip")
    open(zip_path, "wb").write(r.content)
    with zipfile.ZipFile(zip_path) as z:
        z.extractall(tmp)
    gdf = gpd.read_file(tmp)
    gdf = gdf.to_crs("EPSG:4326")
    gdf = gdf[(gdf.STATEFP == statefp) & (gdf.COUNTYFP == countyfp)].copy()
    return gdf

def query_acs_population(state, county, variable):
    params = {
        "get": f"NAME,{variable}",
        "for": "tract:*",
        "in": f"state:{state} county:{county}"
    }
    resp = requests.get(CENSUS_API, params=params)

```



```

resp.raise_for_status()
data = resp.json()
acs = pd.DataFrame(data[1:], columns=data[0])
acs["population"] = acs[variable].astype(int)
acs["GEOID"] = acs.state + acs.county + acs.tract
return acs[["GEOID", "population"]]

def build_coverage_matrix_from_metric(demand_m, candidates_m, radius_m):
    dem_xy = np.c_[demand_m.geometry.x, demand_m.geometry.y]
    cand_xy = np.c_[candidates_m.geometry.x, candidates_m.geometry.y]
    tree = cKDTree(cand_xy)
    neighbors = tree.query_ball_point(dem_xy, r=radius_m)
    n = len(demand_m)
    m = len(candidates_m)
    A = np.zeros((n, m), dtype=int)
    for i, neigh in enumerate(neighbors):
        if len(neigh) > 0:
            A[i, neigh] = 1
    return A

def solve_original_setcover(A, w, p, solver=None, verbose=False):
    """
    A: (n x m) binary numpy
    w: length n (population)
    p: max number of candidate stops to open
    """
    n, m = A.shape

    # Build Gurobi model
    model = gp.Model("setcover")
    model.Params.OutputFlag = 1 if verbose else 0

    # Decision variables
    x = model.addMVar(m, vtype=GRB.BINARY, name="x")
    y = model.addMVar(n, vtype=GRB.BINARY, name="y")

```

```

# Coverage constraints:  $y_i \leq \sum_j A[i,j] x_j$ 
model.addConstrs((y[i] <= A[i, :] @ x for i in range(n)), name="coverage")

# Limit new stops:  $\sum_j x_j \leq p$ 
model.addConstr(x.sum() <= p, name="limit_p")

# Objective: maximize  $w \cdot y$ 
model.setObjective(w @ y, GRB.MAXIMIZE)

# Solve
model.optimize()

if model.Status != GRB.OPTIMAL:
    return None

x_int = (x.X > 0.5).astype(int)
y_int = (y.X > 0.5).astype(int)
return x_int, y_int

# -----

def create_map_and_save(stops, tracts, candidates, demand, chosen,
                        radius_m, max_new, p_weight, scenario_idx, out_dir):
    try:
        import folium
    except Exception:
        print("folium not available, skipping map creation.")
        return

    center = [demand.geometry.y.mean(), demand.geometry.x.mean()]
    m = folium.Map(center, zoom_start=11, tiles="cartodbpositron")

    if "pop_density" in tracts.columns:
        folium.Choropleth(
            geo_data=tracts.to_json(),
            data=tracts,
            columns=["GEOID", "pop_density"],
            key_on="feature.properties.GEOID",

```

```

        fill_color="YlOrRd",
        fill_opacity=0.6,
        line_opacity=0.2,
        legend_name="Population Density"
    ).add_to(m)

    for _, r in stops.iterrows():
        folium.CircleMarker([r.geometry.y, r.geometry.x], radius=2,
            color="gray", fill=True).add_to(m)

    for _, r in candidates.iterrows():
        lat, lon = r.geometry.y, r.geometry.x
        is_chosen = int(r.chosen) == 1 if "chosen" in candidates.columns else
False
        color = "red" if is_chosen else "blue"
        radius = 8 if is_chosen else 4
        popup = folium.Popup(
            f"GEOID:{r.get('GEOID', '')}<br>pop:{r.get('population',
            '')}<br>chosen:{int(is_chosen)}",
            max_width=250
        )
        folium.CircleMarker([lat, lon], radius=radius, color=color, fill=True,
            popup=popup).add_to(m)
        if is_chosen:
            folium.Circle([lat, lon], radius=radius_m, color=color, fill=False,
                opacity=0.4).add_to(m)

    safe_d = str(radius_m).replace('.', '')
    safe_pw = str(p_weight).replace('.', '')
    fname = f"map_s{scenario_idx}_d{safe_d}_n{max_new}_pw{safe_pw}.html"
    out_file = os.path.join(out_dir, fname)
    m.save(out_file)
    print(f"Map saved → {out_file}")
    if scenario_idx == 0:
        webbrowser.open(f"file://{os.path.abspath(out_file)}")

# Printing post optimal maps for potential future use
def create_postoptimal_maps(stops, tracts, candidates, demand, out_dir):
    """
    Reads the post_optimality_equity.csv file and generates
    HTML maps for each scenario stored inside the CSV.

```

```

Mimics the style of create_map_and_save().
"""

# Load scenario results
results_path = os.path.join(out_dir, "post_optimality_equity.csv")
if not os.path.exists(results_path):
    print("No post-optimal results found at:", results_path)
    return

post_df = pd.read_csv(results_path)
if post_df.empty:
    print("Post-optimal CSV is empty.")
    return

print("\nGenerating HTML maps for all post-optimality scenarios...")

# Need folium
try:
    import folium
except Exception:
    print("folium is not installed; cannot create post-optimality maps.")
    return

# Create a fresh metric projection for candidates for rebuild of A
candidates_m = candidates.to_crs(CRS_METRIC)
demand_m = demand.to_crs(CRS_METRIC)

scenario_number = 1

for _, row in post_df.iterrows():
    dmax = row["d_max"]
    pmax = row["max_new"]
    eq = row["equity_mult"]

    # Rebuild A for the scenario
    A_s = build_coverage_matrix_from_metric(demand_m, candidates_m, dmax)

    # Rebuild scenario weights
    pop = demand["population"].values.astype(float)
    if "pop_density" in demand.columns:

```

```

        density = demand["pop_density"].fillna(0).values.astype(float)
    else:
        density = np.zeros(len(demand), dtype=float)

    if density.max() > density.min():
        density_norm = (density - density.min()) / (density.max() -
density.min())
    else:
        density_norm = np.zeros_like(density)

    weights = pop * (1.0 + eq * density_norm)

    # Solve MILP again (same logic as main pipeline)
    sol_s = solve_original_setcover(A_s, weights, pmax, verbose=False)
    if sol_s is None:
        print(f" Scenario {scenario_number}: FAILED to solve")
        scenario_number += 1
        continue

    x_sel, y_sel = sol_s

    # Update candidate flags
    cand_scen = candidates.copy()
    cand_scen["chosen"] = x_sel
    chosen = cand_scen[cand_scen["chosen"] == 1].copy()

    # Create map for this scenario (mimics create_map_and_save)
    center = [demand.geometry.y.mean(), demand.geometry.x.mean()]
    m = folium.Map(center, zoom_start=11, tiles="cartodbpositron")

    # Choropleth
    if "pop_density" in tracts.columns:
        folium.Choropleth(
            geo_data=tracts.to_json(),
            data=tracts,
            columns=["GEOID", "pop_density"],
            key_on="feature.properties.GEOID",
            fill_color="YlOrRd",
            fill_opacity=0.6,
            line_opacity=0.2,

```

```

        legend_name="Population Density"
    ).add_to(m)

# Existing stops
for _, r in stops.iterrows():
    folium.CircleMarker([r.geometry.y, r.geometry.x],
                        radius=2, color="gray", fill=True).add_to(m)

# Candidate stops
for _, r in cand_scen.iterrows():
    lat, lon = r.geometry.y, r.geometry.x
    is_chosen = r.chosen == 1
    color = "red" if is_chosen else "blue"
    radius = 8 if is_chosen else 4
    popup = folium.Popup(
        f"GEOID:{r.get('GEOID','')}<br>"
        f"pop:{r.get('population','')}<br>"
        f"chosen:{int(is_chosen)}",
        max_width=250,
    )
    folium.CircleMarker([lat, lon], radius=radius, color=color,
fill=True,
                        popup=popup).add_to(m)

    if is_chosen:
        folium.Circle([lat, lon], radius=dmax,
                        color=color, fill=False, opacity=0.4).add_to(m)

# Output filename
safe_d = str(dmax).replace(".", "")
safe_eq = str(eq).replace(".", "")
fname =
f"post_map_s{scenario_number}_d{safe_d}_n{pmax}_eq{safe_eq}.html"
out_path = os.path.join(out_dir, fname)

m.save(out_path)
print(f" Scenario {scenario_number}: map saved → {out_path}")

scenario_number += 1

print("\nPost-optimality maps generated.")

```

```

# ----- MAIN PIPELINE -----
def main():
    print("Downloading GTFS stops...")
    stops = download_gtfs_stops(GTFS_URL)

    print("Downloading TIGER tracts and querying ACS population...")
    tracts = download_tiger_tracts(TIGER_TRACTS_URL, FRANKLIN_FIPS[0],
FRANKLIN_FIPS[1])
    acs = query_acs_population(FRANKLIN_FIPS[0], FRANKLIN_FIPS[1],
ACS_VARIABLE)
    tracts = tracts.merge(acs, on="GEOID", how="left")
    tracts["population"] = tracts["population"].fillna(0).astype(int)

    tracts_m = tracts.to_crs(CRS_METRIC)
    tracts["pop_density"] = tracts["population"] / (tracts_m.area / 1e6)
    centroids_m = tracts_m.geometry.centroid
    centroids_wgs84 = centroids_m.to_crs("EPSG:4326")

    demand = tracts.copy()
    demand["geometry"] = centroids_wgs84.values
    demand["lat"] = demand.geometry.y
    demand["lon"] = demand.geometry.x
    demand = gpd.GeoDataFrame(demand, geometry="geometry", crs="EPSG:4326")
    n_dem = len(demand)
    print(f"    Demand points (tract centroids): {n_dem}")

    stops_m = stops.to_crs(CRS_METRIC)
    demand_m = demand.to_crs(CRS_METRIC)

    print("Distance to nearest existing stop...")
    if len(stops_m) == 0:
        demand["dist_to_existing_m"] = np.inf
    else:
        stop_coords = np.c_[stops_m.geometry.x.values,
stops_m.geometry.y.values]
        tree = cKDTree(stop_coords)
        demand_coords = np.c_[demand_m.geometry.x.values,
demand_m.geometry.y.values]

```

```

dists, idxs = tree.query(demand_coords, k=1)
demand["dist_to_existing_m"] = dists

underserved = demand[demand.dist_to_existing_m > D_MAX].copy()
print(f"    Underserved demand points (baseline D_MAX={D_MAX} m):
{len(underserved)}")

candidates = underserved.reset_index(drop=True).copy()
candidates = gpd.GeoDataFrame(candidates, geometry="geometry",
crs="EPSG:4326")
m_cand = len(candidates)
print(f"    Candidate stop locations: {m_cand}")
if m_cand == 0:
    print("No candidate locations (no underserved tracts). Exiting.")
    return

candidates_m = candidates.to_crs(CRS_METRIC)
demand_m = demand.to_crs(CRS_METRIC)

print("Building baseline coverage matrix (KDTree)...")
A_baseline = build_coverage_matrix_from_metric(demand_m, candidates_m,
D_MAX)
w = demand["population"].values.astype(int)

print("Solving MILP with Gurobi...")
sol = solve_original_setcover(A_baseline, w, MAX_NEW_STOPS, verbose=True)
if sol is None:
    raise RuntimeError("Solver failed to find a solution for the base
MILP.")
x_sel, y_sel = sol

candidates["chosen"] = x_sel
demand["covered"] = y_sel

chosen = candidates[candidates["chosen"] == 1].copy()
total_covered_pop = int((w * y_sel).sum())
print("\nSELECTED NEW BUS STOPS (base):")
if not chosen.empty:
    print(chosen[["GEOID", "lat", "lon", "population"]])
else:
    print("    (no candidates selected)")

```



```

print(f"\nTotal population newly covered (approx): {total_covered_pop:,}")

candidates[["GEOID", "lat", "lon", "chosen"]].to_csv(os.path.join(OUT_DIR, "candida
tes.csv"), index=False)

demand[["GEOID", "lat", "lon", "population", "dist_to_existing_m", "covered"]].to_cs
v(os.path.join(OUT_DIR, "demand_coverage.csv"), index=False)

try:
    import folium
    create_map_and_save(stops, tracts, candidates, demand, chosen,
                        radius_m=D_MAX, max_new=MAX_NEW_STOPS,
                        p_weight=1.0, scenario_idx=0, out_dir=OUT_DIR)
except Exception as e:
    print("Skipping base map creation:", e)

print("\n\n=== POST-OPTIMALITY ANALYSIS: exploring parameters ===")

D_MAX_LIST = [600, 805, 1000]
MAX_NEW_LIST = [3, 5, 7]
EQUITY_MULTIPLIERS = [0.0, 1.0, 2.0]

if "pop_density" in demand.columns:
    density = demand["pop_density"].fillna(0).values.astype(float)
else:
    density = np.zeros(len(demand), dtype=float)

if density.max() > density.min():
    density_norm = (density - density.min()) / (density.max() -
density.min())
else:
    density_norm = np.zeros_like(density)

results = []

def run_scenario_rebuild_A_and_solve_with_weights(dmax, max_new, weights):
    A_s = build_coverage_matrix_from_metric(demand_m, candidates_m, dmax)

    sol_s = solve_original_setcover(A_s, weights, max_new, verbose=False)

```

```

        if sol_s is None:
            return None
        x_s, y_s = sol_s
        covered_pop = float((weights * y_s).sum())
        return int(round(covered_pop)), x_s, y_s

scen_idx = 1
for d in D_MAX_LIST:
    for p in MAX_NEW_LIST:
        for e in EQUITY_MULTIPLIERS:
            weights = demand["population"].values * (1.0 + e *
density_norm)

            res = run_scenario_rebuild_A_and_solve_with_weights(d, p,
weights)

            if res is None:
                print(f"Scenario d={d} p={p} e={e} failed to solve.")
                continue
            cov, xvec, yvec = res
            results.append(dict(d_max=d, max_new=p, equity_mult=e,
                                coverage=cov, num_stops=int(xvec.sum())))
            print(f" Scenario {scen_idx}: d_max={d}, p={p}, equity={e} => "
                  f"covered_pop={cov}, stops_used={int(xvec.sum())}")

            cand2 = candidates.copy()
            cand2["chosen"] = xvec
            chosen2 = cand2[cand2.chosen == 1].copy()
            try:
                create_map_and_save(stops, tracts, cand2, demand, chosen2,
                                    radius_m=d, max_new=p, p_weight=e,
                                    scenario_idx=scen_idx, out_dir=OUT_DIR)
            except Exception as e_map:
                print(f" Skipping map for scenario {scen_idx}:", e_map)

            scen_idx += 1

post_df = pd.DataFrame(results)
post_df.to_csv(os.path.join(OUT_DIR, "post_optimality_equity.csv"),
index=False)
print("\nPost-optimality results saved →
output/post_optimality_equity.csv")

```

```

# ----- BEST POST-OPTIMALITY SCENARIO & TOP 5 -----

# Sort by coverage and take top 5
top_post = post_df.sort_values("coverage",
ascending=False).head(5).reset_index(drop=True)

print("\n=== TOP 5 POST-OPTIMALITY SCENARIOS (simplified) ===")
for i, r in top_post.iterrows():
    print(f"\n{i+1}. d_max={r.d_max}, max_new={r.max_new},
equity_mult={r.equity_mult}")
    print(f"    coverage={int(r.coverage)}, stops_used={int(r.num_stops)}")

# ---- Best post-optimal scenario ----
best = top_post.iloc[0]
dmax_best = best.d_max
p_best = best.max_new
eq_best = best.equity_mult

weights_best = demand["population"].values * (1.0 + eq_best * density_norm)
A_best = build_coverage_matrix_from_metric(demand_m, candidates_m,
dmax_best)
sol_best = solve_original_setcover(A_best, weights_best, p_best,
verbose=False)

if sol_best is not None:
    x_best, y_best = sol_best
    cand_best = candidates.copy()
    cand_best["chosen"] = x_best
    chosen_best = cand_best[cand_best.chosen == 1].copy()

print("\nSELECTED NEW BUS STOPS (post-optimal best scenario):")
print(chosen_best[["GEOID", "lat", "lon", "population"]])

# Create and automatically open HTML map for best scenario
try:
    create_map_and_save(
        stops, tracts, cand_best, demand, chosen_best,
        radius_m=dmax_best, max_new=p_best, p_weight=eq_best,
        scenario_idx=0, out_dir=OUT_DIR

```

```
)
except Exception as e_map:
    print("Skipping best scenario map creation:", e_map)

print("\nCreating post-optimality HTML maps...")
create_postoptimal_maps(stops, tracts, candidates, demand, out_dir=OUT_DIR)

# -----
if __name__ == "__main__":
    main()
```

Output

```

Downloading GTFS stops...
Downloading TIGER tracts and querying ACS population...
    Demand points (tract centroids): 328
Distance to nearest existing stop...
    Underserved demand points (baseline D_MAX=805 m): 133
    Candidate stop locations: 133
Building baseline coverage matrix (KDTree)...
Solving MILP with Gurobi...
Set parameter OutputFlag to value 1
Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (win64 - Windows 11.0
(26100.2))

CPU model: AMD Ryzen 7 5800H with Radeon Graphics, instruction set
[SSE2|AVX|AVX2]
Thread count: 8 physical cores, 16 logical processors, using up to 16 threads

Optimize a model with 329 rows, 461 columns and 594 nonzeros
Model fingerprint: 0xdfa76467
Variable types: 0 continuous, 461 integer (461 binary)
Coefficient statistics:
    Matrix range      [1e+00, 1e+00]
    Objective range   [8e+02, 1e+04]
    Bounds range      [1e+00, 1e+00]
    RHS range         [5e+00, 5e+00]
Found heuristic solution: objective -0.0000000
Presolve removed 329 rows and 461 columns
Presolve time: 0.00s
Presolve: All rows and columns removed

Explored 0 nodes (0 simplex iterations) in 0.01 seconds (0.00 work units)
Thread count was 1 (of 16 available processors)

Solution count 2: 42986 -0

Optimal solution found (tolerance 1.00e-04)
Best objective 4.298600000000e+04, best bound 4.298600000000e+04, gap 0.0000%

SELECTED NEW BUS STOPS (base):
      GEOID      lat      lon  population
16  39049009390  39.974217 -82.790753      7768

```

40	39049006241	40.077974	-83.165966	8591
41	39049007398	40.013312	-82.780411	9389
82	39049009755	39.869836	-83.044463	7639
130	39049007209	40.068750	-82.852063	9599

Total population newly covered (approx): 42,986

Map saved → output\map_s0_d805_n5_pw10.html

=== POST-OPTIMALITY ANALYSIS: exploring parameters ===

Scenario 1: d_max=600, p=3, equity=0.0 => covered_pop=27579, stops_used=3

Map saved → output\map_s1_d600_n3_pw00.html

Scenario 2: d_max=600, p=3, equity=1.0 => covered_pop=31107, stops_used=3

Map saved → output\map_s2_d600_n3_pw10.html

Scenario 3: d_max=600, p=3, equity=2.0 => covered_pop=34995, stops_used=3

Map saved → output\map_s3_d600_n3_pw20.html

Scenario 4: d_max=600, p=5, equity=0.0 => covered_pop=42986, stops_used=5

Map saved → output\map_s4_d600_n5_pw00.html

Scenario 5: d_max=600, p=5, equity=1.0 => covered_pop=48501, stops_used=5

Map saved → output\map_s5_d600_n5_pw10.html

Scenario 6: d_max=600, p=5, equity=2.0 => covered_pop=55070, stops_used=5

Map saved → output\map_s6_d600_n5_pw20.html

Scenario 7: d_max=600, p=7, equity=0.0 => covered_pop=57519, stops_used=7

Map saved → output\map_s7_d600_n7_pw00.html

Scenario 8: d_max=600, p=7, equity=1.0 => covered_pop=65335, stops_used=7

Map saved → output\map_s8_d600_n7_pw10.html

Scenario 9: d_max=600, p=7, equity=2.0 => covered_pop=74876, stops_used=7

Map saved → output\map_s9_d600_n7_pw20.html

Scenario 10: d_max=805, p=3, equity=0.0 => covered_pop=27579, stops_used=3

Map saved → output\map_s10_d805_n3_pw00.html

Scenario 11: d_max=805, p=3, equity=1.0 => covered_pop=31107, stops_used=3

Map saved → output\map_s11_d805_n3_pw10.html

Scenario 12: d_max=805, p=3, equity=2.0 => covered_pop=34995, stops_used=3

Map saved → output\map_s12_d805_n3_pw20.html

Scenario 13: d_max=805, p=5, equity=0.0 => covered_pop=42986, stops_used=5

Map saved → output\map_s13_d805_n5_pw00.html

Scenario 14: d_max=805, p=5, equity=1.0 => covered_pop=48501, stops_used=5

Map saved → output\map_s14_d805_n5_pw10.html

Scenario 15: d_max=805, p=5, equity=2.0 => covered_pop=55070, stops_used=5

Map saved → output\map_s15_d805_n5_pw20.html

Scenario 16: d_max=805, p=7, equity=0.0 => covered_pop=57519, stops_used=7
Map saved → output\map_s16_d805_n7_pw00.html

Scenario 17: d_max=805, p=7, equity=1.0 => covered_pop=65335, stops_used=7
Map saved → output\map_s17_d805_n7_pw10.html

Scenario 18: d_max=805, p=7, equity=2.0 => covered_pop=74876, stops_used=7
Map saved → output\map_s18_d805_n7_pw20.html

Scenario 19: d_max=1000, p=3, equity=0.0 => covered_pop=27788, stops_used=3
Map saved → output\map_s19_d1000_n3_pw00.html

Scenario 20: d_max=1000, p=3, equity=1.0 => covered_pop=33639, stops_used=3
Map saved → output\map_s20_d1000_n3_pw10.html

Scenario 21: d_max=1000, p=3, equity=2.0 => covered_pop=39489, stops_used=3
Map saved → output\map_s21_d1000_n3_pw20.html

Scenario 22: d_max=1000, p=5, equity=0.0 => covered_pop=44147, stops_used=5
Map saved → output\map_s22_d1000_n5_pw00.html

Scenario 23: d_max=1000, p=5, equity=1.0 => covered_pop=51700, stops_used=5
Map saved → output\map_s23_d1000_n5_pw10.html

Scenario 24: d_max=1000, p=5, equity=2.0 => covered_pop=59973, stops_used=5
Map saved → output\map_s24_d1000_n5_pw20.html

Scenario 25: d_max=1000, p=7, equity=0.0 => covered_pop=59133, stops_used=7
Map saved → output\map_s25_d1000_n7_pw00.html

Scenario 26: d_max=1000, p=7, equity=1.0 => covered_pop=68927, stops_used=7
Map saved → output\map_s26_d1000_n7_pw10.html

Scenario 27: d_max=1000, p=7, equity=2.0 => covered_pop=79959, stops_used=7
Map saved → output\map_s27_d1000_n7_pw20.html

Post-optimality results saved → output/post_optimality_equity.csv

=== TOP 5 POST-OPTIMALITY SCENARIOS (simplified) ===

1. d_max=1000.0, max_new=7.0, equity_mult=2.0
coverage=79959, stops_used=7
2. d_max=805.0, max_new=7.0, equity_mult=2.0
coverage=74876, stops_used=7
3. d_max=600.0, max_new=7.0, equity_mult=2.0
coverage=74876, stops_used=7
4. d_max=1000.0, max_new=7.0, equity_mult=1.0
coverage=68927, stops_used=7

5. d_max=805.0, max_new=7.0, equity_mult=1.0
coverage=65335, stops_used=7

SELECTED NEW BUS STOPS (post-optimal best scenario):

	GEOID	lat	lon	population
21	39049006932	40.072197	-82.963182	7048
40	39049006241	40.077974	-83.165966	8591
41	39049007398	40.013312	-82.780411	9389
96	39049006945	40.092437	-82.958051	6399
127	39049008164	39.933867	-83.133878	5231
129	39049007199	40.068679	-82.914028	7347
130	39049007209	40.068750	-82.852063	9599

Map saved → output\map_s0_d10000_n7.0_pw20.html

Creating post-optimality HTML maps...

Generating HTML maps for all post-optimality scenarios...

Scenario 1: map saved → output\post_map_s1_d6000_n3.0_eq00.html

Scenario 2: map saved → output\post_map_s2_d6000_n3.0_eq10.html

Scenario 3: map saved → output\post_map_s3_d6000_n3.0_eq20.html

Scenario 4: map saved → output\post_map_s4_d6000_n5.0_eq00.html

Scenario 5: map saved → output\post_map_s5_d6000_n5.0_eq10.html

Scenario 6: map saved → output\post_map_s6_d6000_n5.0_eq20.html

Scenario 7: map saved → output\post_map_s7_d6000_n7.0_eq00.html

Scenario 8: map saved → output\post_map_s8_d6000_n7.0_eq10.html

Scenario 9: map saved → output\post_map_s9_d6000_n7.0_eq20.html

Scenario 10: map saved → output\post_map_s10_d8050_n3.0_eq00.html

Scenario 11: map saved → output\post_map_s11_d8050_n3.0_eq10.html

Scenario 12: map saved → output\post_map_s12_d8050_n3.0_eq20.html

Scenario 13: map saved → output\post_map_s13_d8050_n5.0_eq00.html

Scenario 14: map saved → output\post_map_s14_d8050_n5.0_eq10.html

Scenario 15: map saved → output\post_map_s15_d8050_n5.0_eq20.html

Scenario 16: map saved → output\post_map_s16_d8050_n7.0_eq00.html

Scenario 17: map saved → output\post_map_s17_d8050_n7.0_eq10.html

Scenario 18: map saved → output\post_map_s18_d8050_n7.0_eq20.html

Scenario 19: map saved → output\post_map_s19_d10000_n3.0_eq00.html

Scenario 20: map saved → output\post_map_s20_d10000_n3.0_eq10.html

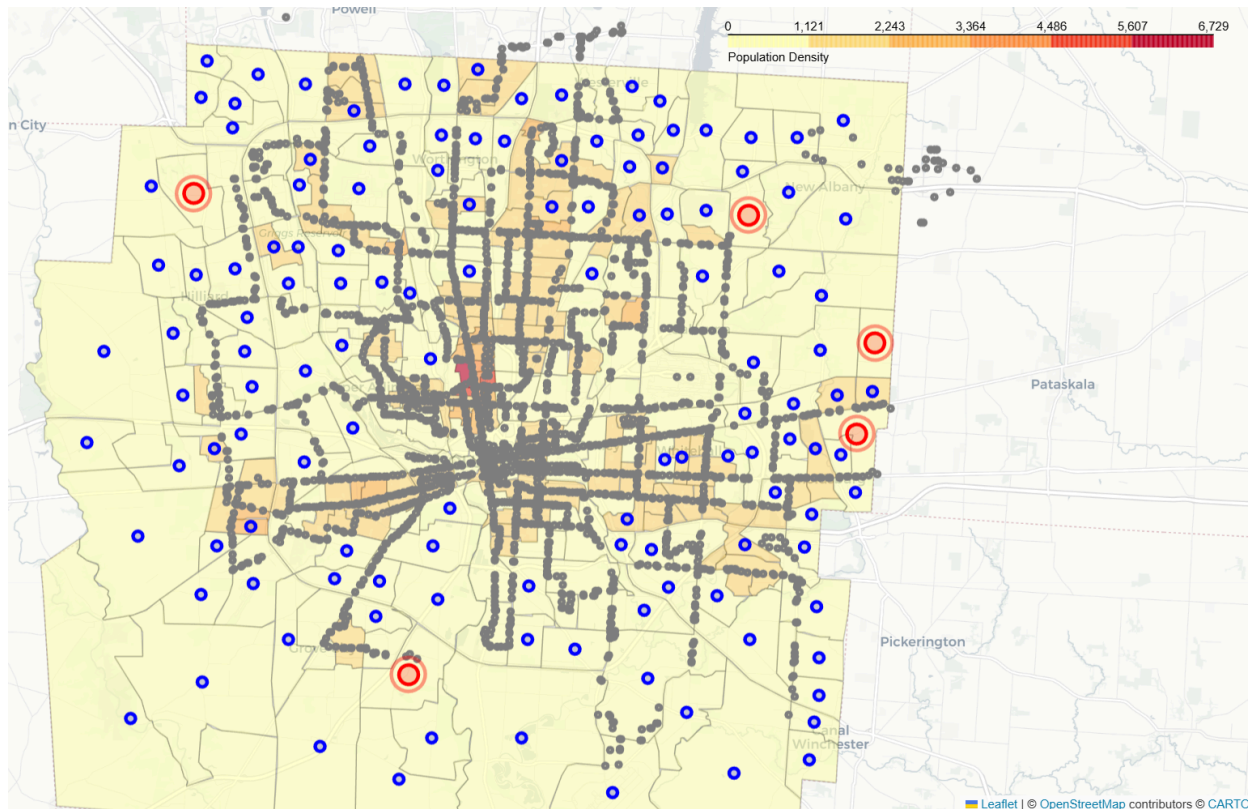
Scenario 21: map saved → output\post_map_s21_d10000_n3.0_eq20.html

Scenario 22: map saved → output\post_map_s22_d10000_n5.0_eq00.html

Scenario 23: map saved → output\post_map_s23_d10000_n5.0_eq10.html
Scenario 24: map saved → output\post_map_s24_d10000_n5.0_eq20.html
Scenario 25: map saved → output\post_map_s25_d10000_n7.0_eq00.html
Scenario 26: map saved → output\post_map_s26_d10000_n7.0_eq10.html
Scenario 27: map saved → output\post_map_s27_d10000_n7.0_eq20.html

Post-optimality maps generated.

Output of best top 5 base scenarios:



Output of best top 7 post optimal scenarios:

